AGILE DEVELOPMENT

What is Agility

- ✓ Agility has become today's buzzword when describing a modern software process. Everyone is agile.
- ✓ An agile team is a nimble team able to appropriately respond to changes. Change is what software development is very much about.
- ✓ Changes in the software being built, changes to the team members, changes because of new technology, changes of all kinds that may have an impact on the product they build or the project that creates the product.
- ✓ Support for changes should be built-in everything we do in software, something we embrace because it is the heart and soul of software.
- ✓ An agile team recognizes that software is developed by individuals working in teams and that the skills of these people, their ability to collaborate, is at the core for the success of the project.
- ✓ Agility is more than an effective response to change. It encourages team structures and attitudes that make communication (among team members, between technologists and businesspeople, between software engineers and their managers) more facile.
- ✓ It emphasizes rapid delivery of operational sounded de-emphasizes the importance of intermediate work products (not always a good thing); it adopts the customer as a part of the development team and works to eliminate the "us and them" attitude that continues to pervade many software projects, it recognizes that planning in an uncertain world has its limits and that a project plan must be flexible. Agility can be applied to any software process.

VTUSYNC.IN

AGILITY AND THE COST OF CHANGE

- ✓ The conventional wisdom in software development (supported by decades of experience) is that the cost of change increases nonlinearly as a project progresses (Figure 3.1, solid black curve).
- ✓ It is relatively easy to accommodate a change when a software team is gathering requirements (early in a project).
- ✓ A usage scenario might have to be modified, a list of functions may be extended, or a written specification can be edited.
- ✓ The costs of doing this work are minimal, and the time required will not adversely affect the outcome of the project.
- ✓ The change requires a modification to the architectural design of the software, the design and construction of three new components, modifications to another five components, the design of new tests, and so on.

- ✓ Costs escalate quickly, and the time and cost required to ensure that the change is made without unintended side effects is nontrivial.
- ✓ Proponents of agility a well-designed agile process "flattens" the cost of change curve (Figure 3.1, shaded, solid curve), allowing a software team to accommodate changes late in a software project without dramatic cost and time impact.

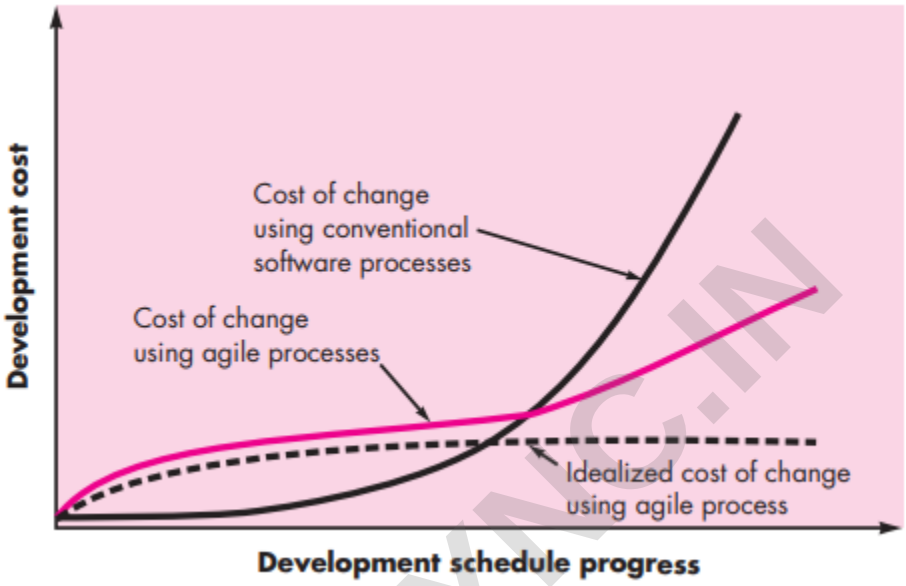


FIGURE 3.1 Change costs as a function of time in development

WHAT I S AN AGILE PROCESS?

- ✓ Any agile software process is characterized in a manner that addresses a few key assumptions about most software projects:
  1. It is difficult to predict in advance which software requirements will persist and which will change. It is equally difficult to predict how customer priorities will change as the project proceeds.
  2. For many types of software, design and construction are interleaved. That is, both activities should be performed in tandem so that design models are proven as they are created. It is difficult to predict how much design is necessary before construction is used to prove the design.
  3. Analysis, design, construction, and testing are not as predictable (from a planning point of view) as we might like.

  Given these three assumptions, an important question arises, how do we create a process that can manage unpredictability?

- ✓ An agile process, therefore, must be adaptable. But continual adaptation without forward progress accomplishes little.

VTUSYNC.IN

- ✓ Therefore, an agile software process must adapt incrementally. To accomplish incremental adaptation, an agile team requires customer feedback (so that the appropriate adaptations can be made).
- ✓ An effective catalyst for customer feedback is an operational prototype or a portion of an operational system. Hence, an incremental development strategy should be instituted. Software increments must be delivered in short time periods so that adaptation keeps pace with change.
- ✓ This iterative approach enables the customer to evaluate the software increment regularly, provide necessary feedback to the software team, and influence the process adaptations that are made to accommodate the feedback.

Agility Principles

The Agile Alliance defines 12 agility principles for those who want to achieve agility:

1. Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
4. Businesspeople and developers must work together daily throughout the project.
5. Build projects around motivated individuals. Give them the environment and support they need and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
7. Working software is the primary measure of progress.
8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
9. Continuous attention to technical excellence and good design enhances agility.
10. Simplicity—the art of maximizing the amount of work not done is essential.
11. The best architectures, requirements, and designs emerge from self– organizing teams.
12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

The Politics of Agile Development

- ✓ There is considerable debate about the benefits and applicability of agile software development as opposed to more conventional software engineering processes.
- ✓ "Traditional methodologists are a bunch of stick-in-the-muds who'd rather produce flawless documentation than a working system that meets business needs".

VTUSYNC.IN

- ✓ Like all software technology arguments, this methodology debate risks degenerating into a religious war. If warfare breaks out, rational thought disappears, and beliefs rather than facts guide decision making.

Human Factors

- ✓ Proponents of agile software development take great pains to emphasize the importance of "people factors."
- ✓ "Agile development focuses on the talents and skills of individuals, molding the process to specific people and teams."
- ✓ The key point in this statement is that the process molds to the needs of the people and team, not the other way around.
- ✓ If members of the software team are to drive the characteristics of the process that is applied to build software, a few key traits must exist among the people on an agile team and the team itself.

Competence.

- ❖ In an agile development context, competence encompasses innate talent, specific software-related skills, and overall knowledge of the process that the team has chosen to apply.
- ❖ Skill and knowledge of process can and should be taught to all people who serve as agile team members.

Common focus.

- ❖ Although members of the agile team may perform different tasks and bring different skills to the project, all should be focused on one goal to deliver a working software increment to the customer within the time promised.
- ❖ To achieve this goal, the team will also focus on continual adaptations (small and large) that will make the process fit the needs of the team.

Collaboration

- ❖ Software engineering is about assessing, analyzing, and using information that is communicated to the software team; creating information that will help all stakeholders understand the work of the team and building information that provides business value for the customer.
- ❖ To accomplish these tasks, team members must collaborate with one another and all other stakeholders.

Decision-making ability.

- ❖ Any good software team (including agile teams) must be allowed the freedom to control its own destiny.

VTUSYNC.IN

- ❖ This implies that the team is given autonomy decision-making authority for both technical and project issues.

` **Fuzzy problem-solving ability**.

- ❖ Software managers must recognize that the agile team will continually have to deal with ambiguity and will continually be buffeted by change.
- ❖ In some cases, the team must accept the fact that the problem they are solving today may not be the problem that needs to be solved tomorrow.

Mutual trust and respect.

- ❖ A jelled team exhibits the trust and respect that are necessary to make them "so strongly knit that the whole is greater than the sum of the parts."

Self-organization.

- ❖ In the context of agile development, self-organization implies three things:
  - I. The agile team organizes itself for the work to be done,
  - II. The team organizes the process to best accommodate its local environment,
  - III. The team organizes the work schedule to best achieve delivery of the software increment.
- ❖ Self-organization has a few technical benefits, but more importantly, it serves to improve collaboration and boost team morale.
- ❖ The team selects how much work it believes it can perform within the iteration, and the team commits to the work. Nothing demotivates a team as much as someone else making commitments to it. Nothing motivates a team as much as accepting the responsibility for fulfilling commitments that it made itself.

VTUSYNC.IN

EXTREME PROGRAMMING (XP)

- ✓ Extreme Programming (XP), the most widely used approach to agile software development.

XP Values

- ✓ Defines a set of five values that establish a foundation for all work performed as part of XP communication, simplicity, feedback, courage, and respect.
- ✓ Each of these values is used as a driver for specific XP activities, actions, and tasks.

- ✓ To achieve effective communication between software engineers and other stakeholders (e.g., to establish required features and functions for the software), XP emphasizes close, yet informal (verbal) collaboration between customers and developers, the establishment of effective metaphors for communicating important concepts, continuous feedback, and the avoidance of voluminous documentation as a communication medium.
- ✓ To achieve simplicity, XP restricts developers to design only for immediate needs, rather than considering future needs.
- ✓ The intent is to create a simple design that can be easily implemented in code). If the design must be improved, it can be refactored later.
- ✓ Feedback is derived from three sources: the implemented software itself, the customer, and other software team members.
- ✓ The degree to which the software implements the output, function, and behavior of the use case is a form of feedback.
- ✓ Finally, as new requirements are derived as part of iterative planning, the team provides the customer with rapid feedback regarding cost and schedule impact.

The XP Process

- ✓ Extreme Programming uses an object-oriented approach as its preferred development paradigm and encompasses a set of rules and practices that occur within the context of four framework activities.
- ✓ Planning, design, coding, and testing. Figure 3.2 illustrates the XP process and notes some of the key ideas and tasks that are associated with each framework activity.

VTUSYNC.IN

Key XP activities are summarized in the paragraphs that follow.

Planning

- ❖ The planning activity (also called the planning game) begins with listening a requirement gathering activity that enables the technical members of the XP team to understand the business context for the software and to get a broad feel for required output and major features and functionality.
- ❖ Listening leads to the creation of a set of “stories” (also called user stories) that describe required output, features, and functionality for software to be built.

- The customer assigns a value (i.e., a priority) to the story based on the overall business value of the feature or function.
- Members of the XP team then assess each story and assign a cost measured in development weeks to it.
- If the story is estimated to require more than three development weeks, the customer is asked to split the story into smaller stories and the assignment of value and cost occurs again.
- It is important to note that new stories can be written at any time. Customers and developers work together to decide how to group stories into the next release (the next software increment) to be developed by the XP team. Once a basic commitment (agreement on stories to be included, delivery date, and other project matters) is made for a release, the XP team orders the stories that will be developed in one of three ways:
  I. All stories will be implemented immediately (within a few weeks),
  II. The stories with highest value will be moved up in the schedule and implemented first, or
  III. The riskiest stories will be moved up in the schedule and implemented first.
- After the first project release (also called a software increment) has been delivered, the XP team computes project velocity.
- Stated simply, project velocity is the number of customer stories implemented during the first release.
- Project velocity can then be used to
  I. Help estimate delivery dates and schedule for subsequent releases and
  II. Determine whether an overcommitment has been made for all stories across the entire development project. If an overcommitment occurs, the content of releases is modified, or end delivery dates are changed.
- As development work proceeds, the customer can add stories, change the value of an existing story, split stories, or eliminate them. The XP team then reconsiders all remaining releases and modifies its plans accordingly

VTUSYNC.IN

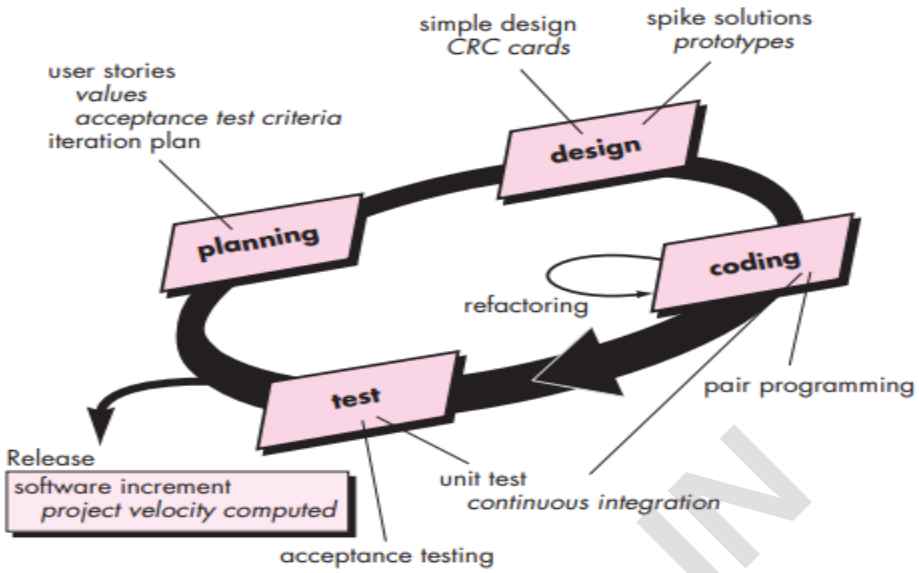


FIGURE 3.2 The Extreme Programming process

Design.

- XP design rigorously follows the KIS (keep it simple) principle.
- A simple design is always preferred over a more complex representation. In addition, the design provides implementation guidance for a story as it is written nothing less, nothing more.
- CRC (class-responsibility collaborator) cards identify and organize the object-oriented classes that are relevant to the current software increment.
- The CRC cards are the only design work product produced as part of the XP process. If a difficult design problem is encountered as part of the design of a story, XP recommends the immediate creation of an operational prototype of that portion of the design. Called a spike solution, the design prototype is implemented and evaluated.
- The intent is to lower risk when true implementation starts and to validate the original estimates for the story containing the design problem.
- Refactoring is the process of changing a software system in such a way that it does not alter the external behavior of the code yet improves the internal structure. It is a disciplined way to clean up code [and modify/simplify the internal design] that minimizes the chances of introducing bugs.
- In essence, when you refactor you are improving the design of the code after it has been written
- Because XP design uses virtually no notation and produces few, if any, work products other than CRC cards and spike solutions, design is viewed as a transient artifact that can and should be continually modified as construction

VTUSYNC.IN

proceeds. A central notion in XP is that design occurs both before and after coding commences.

- ❖ Refactoring means that design occurs continuously as the system is constructed.
- ❖ In fact, the construction activity itself will provide the XP team with guidance on how to improve the design.

Coding.

- ❖ After stories are developed and preliminary design work is done, the team does not move to code, but rather develops a series of unit tests that will exercise each of the stories that is to be included in the current release (software increment).
- ❖ As the individual unit tests are organized into a "universal testing suite", integration and validation testing of the system can occur daily.
- ❖ This provides the XP team with a continual indication of progress and can raise warning flags early if things go awry.
- ❖ "Fixing small problems every few hours takes less time than fixing huge problems just before the deadline."
- ❖ XP acceptance tests, also called customer tests, are specified by the customer and focus on overall system features and functionality that are visible and reviewable by the customer. Acceptance tests are derived from user stories that have been implemented as part of a software release.

Testing.

- ❖ The unit tests that are created should be implemented using a framework that enables them to be automated (hence, they can be executed easily and repeatedly). This encourages a regression testing strategy whenever code is modified (which is often, given the XP refactoring philosophy).
- ❖ As the individual unit tests are organized into a "universal testing suite", integration and validation testing of the system can occur daily.
- ❖ This provides the XP team with a continual indication of progress and can raise warning flags early if things go awry.
- ❖ "Fixing small problems every few hours takes less time than fixing huge problems just before the deadline."
- ❖ XP acceptance tests, also called customer tests, are specified by the customer and focus on overall system features and functionality that are visible and reviewable by the customer.
- ❖ Acceptance tests are derived from user stories that have been implemented as part of a software release.

VTUSYNC.IN

Industrial XP

- Industrial Extreme Programming (IXP) in the following manner: “IXP is an organic evolution of XP. It is imbued with XP’s minimalist, customer-centric, test-driven spirit.
- IXP differs most from the original XP in its greater inclusion of management, its expanded role for customers, and its upgraded technical practices.” IXP incorporates six new practices that are designed to help ensure that an XP project works successfully for significant projects within a large organization.

Readiness assessment

- Prior to the initiation of an IXP project, the organization should conduct a readiness assessment.
- The assessment ascertains whether
  1. an appropriate development environment exists to support IXP,
  2. the team will be populated by the proper set of stakeholders,
  3. the organization has a distinct quality program and supports continuous improvement,
  4. the organizational culture will support the new values of an agile team, and
  5. the broader project community will be populated appropriately.

Project community.

- Classic XP suggests that the right people be used to populate the agile team to ensure success.
- The implication is that people on the team must be well-trained, adaptable and skilled, and have the proper temperament to contribute to a self-organizing team.
- When XP is to be applied for a significant project in a large organization, the concept of the “team” should morph into that of a community.
- A community may have a technologist and customers who are central to the success of a project as well as many other stakeholders (e.g., legal staff, quality auditors, manufacturing or sales types) who “are often at the periphery of an IXP project yet they may play important roles on the project” [Ker05]. In IXP, the community members and their roles should be explicitly defined and mechanisms for communication and coordination between community members should be established.

Project chartering.

VTUSYNC.IN

- ❖ The IXP team assesses the project itself to determine whether an appropriate business justification for the project exists and whether the project will further the overall goals and objectives of the organization.
- ❖ Chartering also examines the context of the project to determine how it complements, extends, or replaces existing systems or processes.

Test-driven management.

- ❖ An IXP project requires measurable criteria for assessing the state of the project and the progress that has been made to date.
- ❖ Establishes a series of measurable "destinations" and then defines mechanisms for determining whether these destinations have been reached.

Retrospectives.

- ❖ An IXP team conducts a specialized technical review after a software increment is delivered.
- ❖ Called a retrospective, the review examines "issues, events, and lessons-learned" across a software increment and/or the entire software release.
- ❖ The intent is to improve the IXP process.

Continuous learning.

- ❖ Because learning is a vital part of continuous process improvement, members of the XP team are encouraged (and possibly, incented) to learn new methods and techniques that can lead to a higher quality product.

- ✓ IXP modifies several existing XP practices. Story-driven development (SDD) insists that stories for acceptance tests be written before a single line of code is generated.
- ✓ Domain-driven design (DDD) is an improvement on the "system metaphor" concept used in XP. DDD [Eva03] suggests the evolutionary creation of a domain model that "accurately represents how domain experts think about their subject".
- ✓ Pairing extends the XP pair programming concept to include managers and other stakeholders.
- ✓ The intent is to improve knowledge sharing among XP team members who may not be directly involved in technical development. Iterative usability discourages front-loaded interface design in favor of usability design that evolves as software increments are delivered and users' interaction with the software is studied.
- ✓ IXP makes smaller modifications to other XP practices and redefines certain roles and responsibilities to make them more amenable to significant projects for large organizations.

VTUSYNC.IN

The XP Debate

- ✓ All new process models and methods spur worthwhile discussion and, in some instances, heated debate. Extreme Programming has done both. In an interesting book that examines the efficacy of XP.
- ✓ Many XP practices are worthwhile, but others have been overhyped, and a few are problematic.
- ✓ The authors suggest that the codependent nature of XP practices are both its strength and its weakness. Because many organizations adopt only a subset of XP practices, they weaken the efficacy of the entire process.
- ✓ Proponents counter that XP is continuously evolving and that many of the issues raised by critics have been addressed as XP practice matures. Among the issues that continue to trouble some critics of XP are

Requirements volatility.

- ❖ Because the customer is an active member of the XP team, changes to requirements are requested informally.
- ❖ Consequently, the scope of the project can change, and earlier work may have to be modified to accommodate current needs. Proponents argue that this happens regardless of the process that is applied and that XP provides mechanisms for controlling scope creep.

Conflicting customer needs.

- ❖ Many projects have multiple customers, each with their own set of needs.
- ❖ In XP, the team itself is tasked with assimilating the needs of different customers, a job that may be beyond their scope of authority.

Requirements are expressed informally.

- ❖ User stories and acceptance tests are the only explicit manifestation of requirements in XP.
- ❖ Critics argue that a more formal model or specification is often needed to ensure that omissions, inconsistencies, and errors are uncovered before the system is built.
- ❖ Proponents counter that the changing nature of requirements makes such models and specifications obsolete almost as soon as they are developed.

Lack of formal design.

- ❖ XP deemphasizes the need for architectural design and in many instances, suggests that design of all kinds should be relatively informal.

VTUSYNC.IN

- ❖ Critics argue that when complex systems are built, design must be emphasized to ensure that the overall structure of the software will exhibit quality and maintainability.
- ❖ XP proponents suggest that the incremental nature of the XP process limits complexity (simplicity is a core value) and therefore reduces the need for extensive design.

OTHER AGILE PROCESS MODELS

- ✓ The history of software engineering is littered with dozens of obsolete process descriptions and methodologies, modeling methods and notations, tools, and technology.
- ✓ Each flared in notoriety and was then eclipsed by something new and (purportedly) better. With the introduction of a wide array of agile process models each contending for acceptance within the software development community the agile movement is following the same historical path.
- ✓ The most widely used of all agile process models is Extreme Programming (XP). But many other agile process models have been proposed and are in use across the industry.
- ✓ Among the most common are:
  1. Adaptive Software Development (ASD)
  2. Scrum
  3. Dynamic Systems Development Method (DSDM)
  4. Crystal
  5. Feature Drive Development (FDD)
  6. Lean Software Development (LSD)
  7. Agile Modeling (AM)
  8. Agile Unified Process (AUP)

VTUSYNC.IN

1. Adaptive Software Development (ASD)

- ✓ Adaptive Software Development (ASD) has been proposed by Jim Highsmith as a technique for building complex software and systems.
- ✓ The philosophical underpinnings of ASD focus on human collaboration and team self-organization.
- ✓ Highsmith argues that an agile, adaptive development approach based on collaboration is “as much a source of order in our complex interactions as discipline and engineering.”
- ✓ He defines an ASD “life cycle” as shown in Figure 3.3 that incorporates three phases, speculation, collaboration, and learning

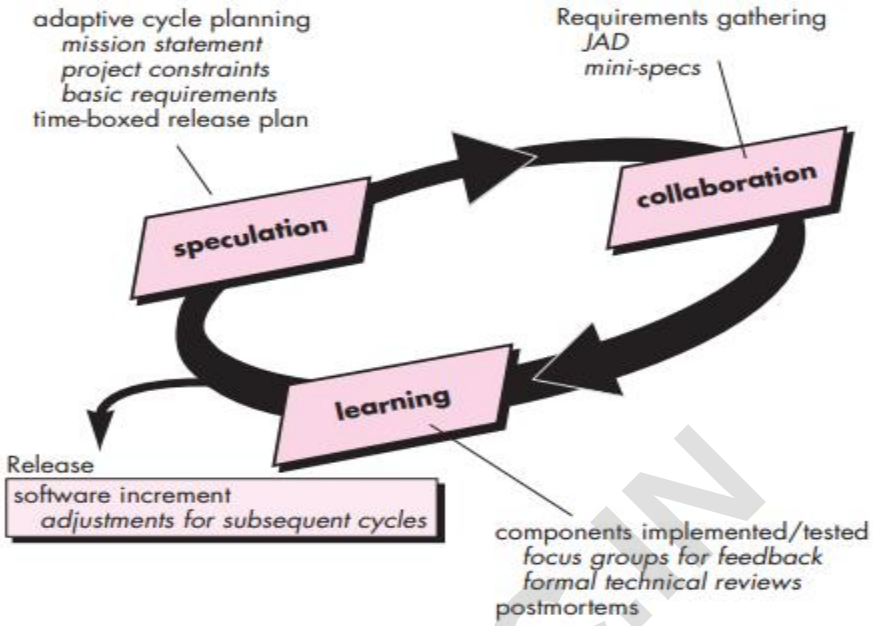


FIGURE 3.3 Adaptive software development

- ✓ During speculation, the project is initiated, and adaptive cycle planning is conducted. Adaptive cycle planning uses project initiation information the customer's mission statement, project constraints (e.g., delivery dates or user descriptions), and basic requirements to define the set of release cycles (software increments) that will be required for the project.
- ✓ No matter how complete and farsighted the cycle plan, it will invariably change.
- ✓ Based on information obtained at the completion of the first cycle, the plan is reviewed and adjusted so that planned work better fits the reality in which an ASD team is working.
- ✓ Motivated people use collaboration in a way that multiplies their talent and creative output beyond their absolute numbers.
- ✓ This approach is a recurring theme in all agile methods. But collaboration is not easy. It encompasses communication and teamwork, but it also emphasizes individualism, because individual creativity plays an important role in collaborative thinking.
- ✓ It is, above all, a matter of trust. People working together must trust one another to

  i. Criticize without animosity,
  ii. Assist without resentment,
  iii. Work as hard as or harder than they do,
  iv. Have the skill set to contribute to the work at hand, and

VTUSYNCIN

v. Communicate problems or concerns in a way that leads to effective action.

- ✓ As members of an ASD team begin to develop the components that are part of an adaptive cycle, the emphasis is on "learning" as much as it is on progress toward a completed cycle.
- ✓ The ASD philosophy has merit regardless of the process model that is used. ASD's overall emphasis on the dynamics of self-organizing teams, interpersonal collaboration, and individual and team learning yield software project teams that have a much higher likelihood of success.

2. Scrum

- ✓ Scrum is an agile software development method that was conceived by Jeff Sutherland and his development team in the early 1990s.
- ✓ Scrum principles are consistent with the agile manifesto and are used to guide development activities within a process that incorporates the following framework activities: **requirements, analysis, design, evolution, and delivery**.
- ✓ Within each framework activity, work tasks occur within a process pattern called a sprint.
- ✓ The work conducted within a sprint (the number of sprints required for each framework activity will vary depending on product complexity and size) is adapted to the problem at hand and is defined and often modified in real time by the Scrum team.
- ✓ The overall flow of the Scrum process is illustrated in Figure 3.4.
- ✓ Scrum emphasizes the use of a set of software process patterns that have proven effective for projects with tight timelines, changing requirements, and business criticality.
- ✓ Each of these process patterns defines a set of development actions:
- ✓ **Backlog**: - a prioritized list of project requirements or features that provide business value for the customer. Items can be added to the backlog at any time. The product manager assesses the backlog and updates priorities as required.
- ✓ **Sprints: -** consist of work units that are required to achieve a requirement defined in the backlog that must be fit into a predefined time-box(typically 30 days). Changes (e.g., backlog work items) are not introduced during the sprint. Hence, the sprint allows team members to work in a short-term, but stable environment.

VTUSYNC.IN

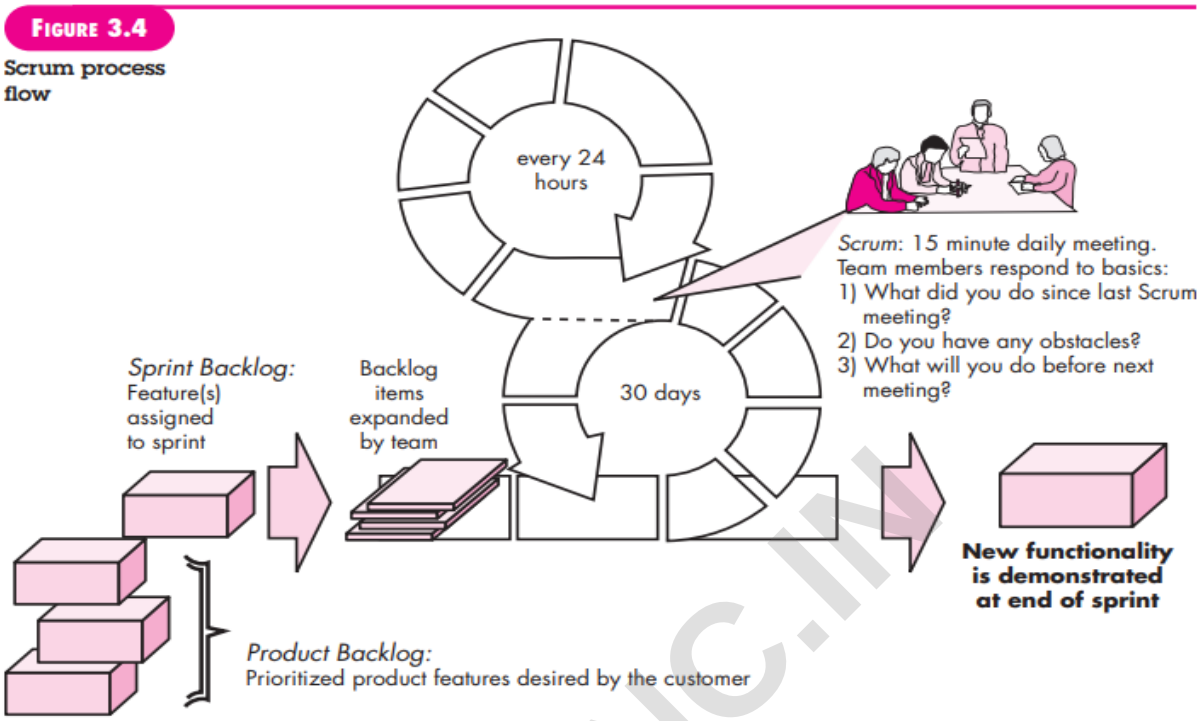


FIGURE 3.4
Scrum process flow

- ✓ **Scrum meetings**: - Are short (typically 15 minutes) meetings held daily by the Scrum team. Three key questions are asked and answered by all team members
  - ❖ What did you do since the last team meeting?
  - ❖ What obstacles are you encountering?
  - ❖ What do you plan to accomplish by the next team meeting?
- ✓ A team leader, called **a Scrum master,** leads the meeting and assesses the responses from each person.
- ✓ The Scrum meeting helps the team to uncover potential problems as early as possible. Also, these daily meetings lead to "knowledge socialization" and thereby promote a self-organizing team structure.
- ✓ **Demos:- D**eliver the software increment to the customer so that functionality that has been implemented can be demonstrated and evaluated by the customer. It is important to note that the demo may not contain all the planned functionality, but rather those functions that can be delivered within the time-box that was established.

3. Dynamic Systems Development Method (DSDM)

- ✓ The Dynamic Systems Development Method (DSDM) is an agile software development approach that "provides a framework for building and maintaining systems which meet tight time constraints through the use of incremental prototyping in a controlled project environment" .

VTUSYNC.IN

- ✓ The DSDM philosophy is borrowed from a modified version of the Pareto principle 80 percent of an application can be delivered in 20 percent of the time it would take to deliver the complete (100 percent) application.
- ✓ DSDM is an iterative software process in which each iteration follows the 80 percent rule. That is, only enough work is required for each increment to facilitate movement to the next increment.
- ✓ The remaining detail can be completed later when more business requirements are known or changes have been requested and accommodated.
- ✓ The DSDM Consortium is a worldwide group of member companies that collectively take on the role of "keeper" of the method.
- ✓ The consortium has defined an agile process model, called the DSDM life cycle that defines three **different iterative cycles, preceded by two additional life cycle activities:**
- ✓ **Feasibility study**:- Establishes the basic business requirements and constraints associated with the application to be built and then assesses whether the application is a viable candidate for the DSDM process.
- ✓ **Business study:-** Establishes the functional and information requirements that will allow the application to provide business value; also, defines the basic application architecture and identifies the maintainability requirements for the application.
- ✓ **Functional model iteration: - P**roduces a set of incremental prototypes that demonstrate functionality for the customer. The intent during this iterative cycle is to gather additional requirements by eliciting feedback from users as they exercise the prototype.
- ✓ **Design and build iteration: - R**evisits prototypes built during functional model iteration to ensure that each has been engineered in a manner that will enable it to provide operational business value for end users. In some cases, functional model iteration and design and build iteration occur concurrently.

✓ **Implementation: - P**laces the latest software increment into the operational environment. It should be noted that

i. The increment may not be 100 percent complete or

ii. Changes may be requested as the increment is put into place. In either case, DSDM development work continues by returning to the functional model iteration activity.

4. Crystal

- ✓ A primary goal of delivering useful, working software and a secondary goal of setting up for the next.
- ✓ To achieve maneuverability, defined a set of methodologies, each with core elements that are common to all, and roles, process patterns, work products, and practice that are unique to each.

VTUSYNC.IN

- ✓ The Crystal family is a set of example agile processes that have been proven effective for different types of projects.
- ✓ The intent is to allow agile teams to select the member of the crystal family that is most appropriate for their project and environment.

5. Feature Driven Development (FDD)

- ✓ Feature Driven Development (FDD) was originally conceived by Peter Coad and his colleagues as a practical process model for object-oriented software engineering.
- ✓ Like other agile approaches, FDD adopts a philosophy that

  i. Emphasizes collaboration among people on an FDD team
  ii. Manages problem and project complexity using feature-based decomposition followed by the integration of software increments, and
  iii. Communication of technical detail using verbal, graphical, and text-based means.

- ✓ FDD emphasizes software quality assurance activities by encouraging an incremental development strategy, the use of design and code inspections, the application of software quality assurance audits the collection of metrics, and the use of patterns (for analysis, design, and construction).
- ✓ The emphasis on the definition of features provides the following benefits:
  - ❖ Because features are small blocks of deliverable functionality, users can describe them more easily; understand how they relate to one another more readily; and better review them for ambiguity, error, or omissions.
  - ❖ Features can be organized into a hierarchical business-related grouping.
  - ❖ Since a feature is the FDD deliverable software increment, the team develops operational features every two weeks.
  - ❖ Because features are small, their design and code representations are easier to inspect effectively.
  - ❖ Project planning, scheduling, and tracking are driven by the feature hierarchy, rather than an arbitrarily adopted software engineering task set. Coad and his colleagues suggest the following template for defining a feature:

<action> the **<result>** **<by | for | of | to>** a(n) **<object>**

VTUSYNC.IN

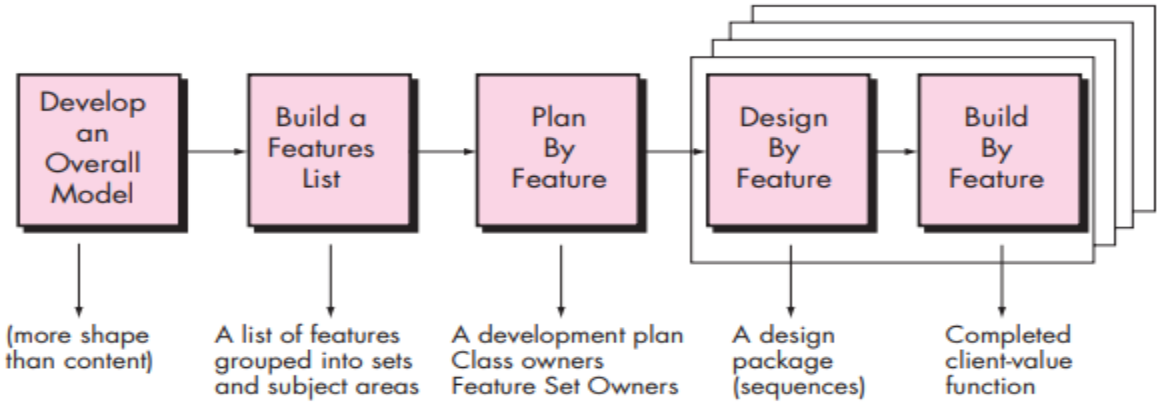


FIGURE 3.5 Feature Driven Development [Coa99] (with permission)

where an **<object> is** "a person, place, or thing." Examples of features for an e-commerce application might be:

- ❖ Add the product to shopping cart
- ❖ Display the technical-specifications of the product
- ❖ Store the shipping-information for the customer

- ✓ For example: Making a product sale is a feature set that would encompass the features noted earlier and others.
- ✓ The FDD approach defines five "collaborating" framework activities as shown in Figure 3.5.
- ✓ FDD provides greater emphasis on project management guidelines and techniques than many other agile methods. As projects grow and complexity, ad hoc project management is often inadequate.
- ✓ It is essential for developers, their managers, and other stakeholders to understand project status what accomplishments have been made and problems have been encountered.
- ✓ If deadline pressure is significant, it is critical to determine if software increments (features) are properly scheduled.
- ✓ To accomplish this, FDD defines six milestones during the design and implementation of a feature: "**design walkthrough, design, design inspection, code, code inspection, promote to build**"

6. Lean Software Development (LSD)

- ✓ Lean Software Development (LSD) has adapted the principles of lean manufacturing to the world of software engineering.
- ✓ The lean principles that inspire the LSD process can be as eliminate waste, build quality in, create knowledge, defer commitment, deliver fast, respect people, and optimize the whole.
- ✓ Each of these principles can be adapted to the software process.

VTUSYNC.IN

- ✓ For example, eliminate waste within the context of an agile software project can be interpreted to mean
  - i. Adding no extraneous features or functions,
  - ii. Assessing the cost and schedule impact of any newly requested requirement,
  - iii. Removing any superfluous process steps,
  - iv. Establishing mechanisms to improve the way team members find information
  - v. Ensuring the testing finds as many errors as possible,
  - vi. Reducing the time required to request and get a decision that affects the software or the process that is applied to create it, and
  - vii. Streamlining the way information is transmitted to all stakeholders involved in the process.

7. Agile Modeling (AM)

- ✓ There are many situations in which software engineers must build large, business-critical systems.
- ✓ The scope and complexity of such systems must be modeled so that
  - i. All constituencies can better understand what needs to be accomplished,
  - ii. The problem can be partitioned effectively among the people who must solve it, and
  - iii. Quality can be assessed as the system is being engineered and built.
- ✓ Over the past 30 years, a wide variety of software engineering modeling methods and notation have been proposed for analysis and design.
- ✓ These methods have merit, but they have proven to be difficult to apply and challenging to sustain (over many projects). Part of the problem is the “weight” of these modeling methods.
- ✓ Analysis and design modeling have substantial benefit for large projects if for no other reason than to make these projects intellectually manageable.
- ✓ Agile Modeling (AM) is a collection of values, principles, and practices for modeling software that can be applied on a software development project in an effective and light-weight manner.
- ✓ Agile models are more effective than traditional models because they are just barely good, they don’t have to be perfect.
- ✓ Agile modeling adopts all of the values that are consistent with the agile manifesto.
- ✓ The agile modeling philosophy recognizes that an agile team must have the courage to make decisions that may cause it to reject a design and refactor.
- ✓ The team must also have the humility to recognize that technologists do not have all the answers and that business experts and other stakeholders should be respected and embraced

VTUSYNC.IN

- ✓ Although AM suggests a wide array of "core" and "supplementary" modeling principles, those that make AM unique are
- ✓ **Model with a purpose: -** A developer who uses AM should have a specific goal in mind before creating the model. Once the goal for the model is identified, the type of notation to be used and level of detail required will be more obvious.
- ✓ **Use multiple models**: - There are many different models and notations that can be used to describe software. Only a small subset is essential for most projects. AM suggests that to provide needed insight, each model should present a different aspect of the system and only those models that provide value to their intended audience should be used.
- ✓ **Travel light**: - As software engineering work proceeds, keep only those models that will provide long-term value and jettison the rest. Every work product that is kept must be maintained as changes occur. This represents work that slows the team down.
- ✓ **Content is more important than representation: -** Modeling should impart information to its intended audience. A syntactically perfect model that imparts little useful content is not as valuable as a model with flawed notation that nevertheless provides valuable content for its audience.
- ✓ **Know the models and the tools you use to create them:-** Understand the strengths and weaknesses of each model and the tools that are used to create it.
- ✓ **Adapt locally:-** The modeling approach should be adapted to the needs of the agile team.

8. Agile Unified Process (AUP)

- ✓ The Agile Unified Process (AUP) adopts a "serial in the large" and "iterative in the small" philosophy for building computer-based systems.
- ✓ By adopting the classic UP phased activities **inception, elaboration, construction, and transition**
- ✓ AUP provides a serial overlay that enables a team to visualize the overall process flow for a software project. However, within each of the activities, the team iterates to achieve agility and to deliver meaningful software increments to end users as rapidly as possible.
- ✓ Each AUP iteration addresses the following activities
  - ❖ **Modeling.** UML representations of the business and problem domains are created. However, to stay agile, these models should be "just barely good enough" to allow the team to proceed.
  - ❖ ***Implementation.*** Models are translated into source code
  - ❖ **Testing.** Like XP, the team designs and executes a series of tests to uncover errors and ensure that the source code meets its requirements.
  - ❖ **Deployment.** In this context focuses on the delivery of a software increment and the acquisition of feedback from end users.

VTUSYNC.IN

- **Configuration and project management**. In the context of AUP, configuration management addresses change management, risk management, and the control of any persistent work products17 that are produced by the team. Project management tracks and controls the progress of the team and coordinates team activities.
- **Environment management.** Environment management coordinates a process infrastructure that includes standards, tools, and other support technology available to the team. Although the AUP has historical and technical connections to the Unified Modeling Language, it is important to note that UML modeling can be using in conjunction with any of the agile process models

TOOL SET FOR THE AGILE PROCESS

- ✓ Some proponents of the agile philosophy argue that automated software tools (e.g., design tools) should be viewed as a minor supplement to the team's activities, and not at all pivotal to the success of the team.
- ✓ Tools can have a benefit and that "agile teams stress using tools that permit the rapid flow of understanding. Some of those tools are social, starting even at the hiring stage.
- ✓ Some tools are technological, helping distributed teams simulate being physically present.
- ✓ Many tools are physical, allowing people to manipulate them in workshops." Because acquiring the right people (hiring), team collaboration, stakeholder communication, and indirect management are key elements in virtually all agile process models, Cockburn argues that "tools" that address these issues are critical success factors for agility. For example, a hiring "tool" might be the requirement to have a prospective team member spend a few hours pair programming with an existing member of the team.

MODELING

SOFTWARE ENGINEERING KNOWLEDGE

- ✓ Many software practitioners think of software engineering knowledge almost exclusively as knowledge of specific technologies: Java, Perl, html, C, Linux, Windows NT, and so on.
- ✓ Knowledge of specific technological details is necessary to perform computer programming.
- ✓ If someone assigns you to write a program in C, you must know something about C to get your program to work.

VTUSYNC.IN

- ✓ Often hear people say that software development knowledge has a 3-year half-life: half of what you need to know today will be obsolete within 3 years.
- ✓ In the domain of technology-related knowledge, that's probably about right. But there is another kind of software development knowledge, a kind that I think of as "software engineering principles" that does not have a three-year half-life.
- ✓ These software engineering principles are likely to serve a professional programmer throughout his or her career.

CORE PRINCIPLES

- ✓ Software engineering is guided by a collection of core principles that help in the application of a meaningful software process and the execution of effective software engineering methods.
- ✓ **At the process level**, core principles establish a philosophical foundation that guides a software team as it performs framework and umbrella activities, navigates the process flow, and produces a set of software engineering work products.
- ✓ **At the level of practice**, core principles establish a collection of values and rules that serve as a guide as analyze a problem, design a solution, implement and test the solution, and ultimately deploy the software in the user community.
- ✓ The general principles that span software engineering process and practice:

1. Provide value to end users,
2. Keep it simple,
3. Maintain the vision (of the product and the project),
4. Recognize that others consume (and must understand) what you produce,
5. Be open to the future,
6. Plan for reuse, and
7. Think! Although these general principles are important, they are characterized at such a high level of abstraction that they are sometimes difficult to translate into day-to-day software engineering practice. In the subsections that follow, I take a more detailed look at the core principles that guide process and practice.

Principles That Guide Process

- ✓ The following set of core principles can be applied to the framework, and by extension, to every software process.

Principle 1:- Be agile. Whether the process model you choose is prescriptive or agile, the basic tenets of agile development should govern your approach. Every aspect of the work you do should emphasize economy of action keep your technical approach as simple as possible, keep the work products you produce as concise as possible, and make decisions locally whenever possible.

VTUSYNC.IN

Principle 2:- Focus on quality at every step. The exit condition for every process activity, action, and task should focus on the quality of the work product that has been produced.

Principle 3:- Be ready to adapt. Process is not a religious experience, and dogma has no place in it. When necessary, adapt your approach to constraints imposed by the problem, the people, and the project itself.

Principle 4: - Build an effective team. Software engineering process and practice are important, but the bottom line is people. Build a self-organizing team that has mutual trust and respect.

Principle 5: - Establish mechanisms for communication and coordination. Projects fail because important information falls into the cracks and/or stakeholders fail to coordinate their efforts to create a successful product. These are management issues, and they must be addressed.

Principle 6: - Manage change. The approach may be either formal or informal, but mechanisms must be established to manage the way changes are requested, assessed, approved, and implemented.

Principle 7: - Assess risk. Lots of things can go wrong as software is being developed. It's essential that you establish contingency plans.

Principle 8: - Create work products that provide value for others. Create only those work products that provide value for other process activities, actions, or tasks. Every work product that is produced as part of software engineering practice will be passed on to someone else.

VTUSYNC.IN

Principles That Guide Practice

- ✓ Software engineering practice has a single overriding goal to deliver on-time, high-quality, operational software that contains functions and features that meet the needs of all stakeholders.
- ✓ To achieve this goal, adopt a set of core principles that guide the technical work.
- ✓ These principles have merit regardless of the analysis and design methods that can be applied, the construction techniques (e.g., programming language, automated tools) that can be used, or the verification and validation approach that can be chosen.
- ✓ The following set of core principles are fundamental to the practice of software engineering:

Principle 1. Divide and conquer. Stated in a more technical manner, analysis and design should always emphasize separation of concerns (SoC). A large problem is easier to solve if it is subdivided into a collection of elements (or concerns). Ideally, each concern delivers distinct functionality that can be developed, and in some cases validated, independently of other concerns.

Principle 2. Understand the use of abstraction. At its core, an abstraction is a simplification of some complex element of a system used to communicate meaning in a single phrase. In software engineering practice, may use many different levels of abstraction, each imparting or implying meaning that must be communicated. In analysis and design work, a software team normally begins with models that represent high levels of abstraction and slowly refines those models into lower levels of abstraction.

Principle 3. Strive for consistency. Whether it's creating a requirements model, developing a software design, generating source code, or creating test cases, the principle of consistency suggests that a familiar context makes software easier to use. As an example, consider the design of a user interface for a WebApp. Consistent placement of menu options, the use of a consistent color scheme, and the consistent use of recognizable icons all help to make the interface ergonomically sound.

Principle 4. Focus on the transfer of information. Software is about information transfer from a database to an end user, from a legacy system to a WebApp, from an end user into a graphic user interface (GUI), from an operating system to an application, from one software component to another the list is almost endless. In every case, information flows across an interface, and consequently, there are opportunities for error, or omission, or ambiguity.

Principle 5. Build software that exhibits effective modularity. Modularity provides a mechanism for realizing the philosophy. Any complex system can be divided into modules (components), but good software engineering practice demands more. Modularity must be effective. That is, each module should focus exclusively on one well-constrained aspect of the system it should be cohesive in its function and/or constrained in the content it represents.

Principle 6. The goal of patterns within the software community is to create a body of literature to help software developers resolve recurring problems encountered throughout all of software development. Patterns help create a shared language for communicating insight and experience about these problems and their

VTUSYNC.IN

solutions. Formally codifying these solutions and their relationships lets us successfully capture the body of knowledge which defines our understanding of good architectures that meet the needs of their users.

Principle 7. When possible, represent the problem and its solution from several different perspectives. When a problem and its solution are examined from a number of different perspectives, it is more likely that greater insight will be achieved, and that errors and omissions will be uncovered.

Principle 8. Remember that someone will maintain the software. Over the long term, software will be corrected as defects are uncovered, adapted as its environment changes, and enhanced as stakeholders request more capabilities. These maintenance activities can be facilitated if solid software engineering practice is applied throughout the software process.

PRINCIPLES THAT GUIDE EACH FRAMEWORK ACTIVITY

Communication Principles

- ✓ Before customer requirements can be analyzed, modeled, or specified they must be gathered through the communication activity.
- ✓ A customer has a problem that may be amenable to a computer-based solution. You respond to the customer's request for help.
- ✓ Communication has begun. But the road from communication to understanding is often full of potholes.
- ✓ Effective communication (among technical peers, with the customer and other stakeholders, and with project managers) is among the most challenging activities that will confront.

Principle 1. Listen. Try to focus on the speaker's words, rather than formulating your response to those words. Ask for clarification if something is unclear but avoid constant interruptions. Never become contentious in words or actions (e.g., rolling your eyes or shaking your head) as a person is talking.

Principle 2. Prepare before you communicate. Spend the time to understand the problem before you meet with others. If necessary, do some research to understand business domain jargon. If you have responsibility for conducting a meeting, prepare an agenda in advance of the meeting.

Principle 3. Someone should facilitate the activity. Every communication meeting should have a leader

- ❖ To keep the conversation moving in a productive direction,

VTUSYNC.IN

- ❖ To mediate any conflict that does occur, and
- ❖ To ensure than other principles are followed.

Principle 4. Face-to-face communication is best. But it usually works better when some other representation of the relevant information is present.

Principle 5. Take notes and document decisions. Things have a way of falling into the cracks. Someone participating in the communication should serve as a "recorder" and write down all important points and decisions.

Principle 6. Strive for collaboration. Collaboration and consensus occur when the collective knowledge of members of the team is used to describe product or system functions or features. Each small collaboration serves to build trust among team members and creates a common goal for the team.

Principle 7. Stay focused; modularize your discussion. The more people involved in any communication, the more likely that discussion will bounce from one topic to the next. The facilitator should keep the conversation modular, leaving one topic only after it has been resolved (however, see Principle 9).

Principle 8. If something is unclear, draw a picture. Verbal communication goes only so far. A sketch or drawing can often provide clarity when words fail to do the job.

Principle 9. (a) Once you agree to something, move on. (b) If you can't agree to something, move on. (c) If a feature or function is unclear and cannot be clarified at the moment, move on. Communication, like any software engineering activity, takes time. Rather than iterating endlessly, the people who participate should recognize that many topics require discussion (see Principle 2) and that "moving on" is sometimes the best way to achieve communication agility.

Principle 10. Negotiation is not a contest or a game. It works best when both parties win. There are many instances in which you and other stakeholders must negotiate functions and features, priorities, and delivery dates. If the team has collaborated well, all parties have a common goal. Still, negotiation will demand compromise from all parties.

Planning Principles

- ✓ The communication activity helps you to define your overall goals and objectives (subject, of course, to change as time passes).
- ✓ However, understanding these goals and objectives is not the same as defining a plan for getting there.
- ✓ The planning activity encompasses a set of management and technical practices that enable the software team to define a road map as it travels toward its strategic

VTUSYNC.IN

goal and tactical objectives. Try as we might, it's impossible to predict exactly how a software project will evolve. There is no easy way to determine what unforeseen technical problems will be encountered, what important information will remain undiscovered until late in the project, what misunderstandings will occur, or what business issues will change. And yet, a good software team must plan its approach.

- ✓ Regardless of the rigor with which planning is conducted, the following principles always apply:

Principle 1. ***Understand the scope of the project.*** It's impossible to use a road map if don't know where you're going. Scope provides the software team with a destination.

Principle 2. ***Involve stakeholders in the planning activity***. Stakeholders define priorities and establish project constraints. To accommodate these realities, software engineers must often negotiate order of delivery, timelines, and other project-related issues.

Principle 3. ***Recognize that planning is iterative.*** A project plan is never engraved in stone. As work begins, it is very likely that things will change. Therefore, the plan must be adjusted to accommodate these changes. In addition, iterative, incremental process models dictate replanning after the delivery of each software increment based on feedback received from users.

Principle 4. ***Estimate based on what you know.*** The intent of estimation is to provide an indication of effort, cost, and task duration, based on the team's current understanding of the work to be done. If information is vague or unreliable, estimates will be equally unreliable.

Principle 5. ***Consider risk as you define the plan***. If you have identified risks that have high impact and high probability, contingency planning is necessary. In addition, the project plan should be adjusted to accommodate the likelihood that one or more of these risks will occur

Principle 6. Be realistic. People don't work 100 percent of every day. Noise always enters into any human communication. Omissions and ambiguity are facts of life. Change will occur. Even the best software engineers make mistakes. These and other realities should be considered as a project plan is established.

Principle 7. Adjust granularity as you define the plan. Granularity refers to the level of detail that is introduced as a project plan is developed. A "high granularity" plan provides significant work task detail that is planned over relatively short time increments (so that tracking and control occur frequently). A "low granularity" plan provides broader work tasks that are planned over longer time periods. In general,

VTUSYNC.IN

granularity moves from high to low as the project timeline moves away from the current date.

Principle 8. **Define how you intend to ensure quality.** The plan should identify how the software team intends to ensure quality. If technical reviews3 are to be conducted, they should be scheduled. If pair programming is to be used during construction, it should be explicitly defined within the plan.

Principle 9. Describe how you intend to accommodate change. Even the best planning can be obviated by uncontrolled change. should identify how changes are to be accommodated as software engineering work proceeds.

Modeling Principles

- ✓ We create models to gain a better understanding of the actual entity to be built.
- ✓ When the entity is a physical thing (e.g., a building, a plane, a machine), we can build a model that is identical in form and shape but smaller in scale.
- ✓ However, when the entity to be built is software, our model must take a different form. It must be capable of representing the information that software transforms, the architecture and functions that enable the transformation to occur, the features that users desire, and the behavior of the system as the transformation is taking place.
- ✓ Models must accomplish these objectives at different levels of abstraction first depicting the software from the customer's viewpoint and later representing the software at a more technical level.
- ✓ In software engineering work, two classes of models can be created: requirements models and design models.
- ✓ Requirements models (also called analysis models) represent customer requirements by depicting the software in three different domains: the information domain, the functional domain, and the behavioral domain.
- ✓ Design models represent characteristics of the software that help practitioners to construct it effectively: the architecture, the user interface, and component-level detail.
- ✓ All software engineers who perform modeling actions and tasks:

Principle 1. The primary goal of the software team is to build software, not create models. Agility means getting software to the customer in the fastest possible time. Models that make this happen are worth creating, but models that slow the process down or provide little new insight should be avoided.

Principle 2. Travel light—don't create more models than you need. Every model that is created must be kept up-to-date as changes occur. More importantly, every new model takes time that might otherwise be spent on construction (coding and testing).

VTUSYNC.IN

Therefore, create only those models that make it easier and faster to construct the software.

Principle 3. Strive to produce the simplest model that will describe the problem or the software. By keeping models simple, the resultant software will also be simple. The result is software that is easier to integrate, easier to test, and easier to maintain. In addition, simple models are easier for members of the software team to understand and critique, resulting in an ongoing form of feedback that optimizes the result.

Principle 4. Build models in a way that makes them amenable to change. Assume that models will change, but in making this assumption don't get sloppy.

Principle 5. Be able to state an explicit purpose for each model that is created. Every time create a model, ask why to need to do this. If you can't provide solid justification for the existence of the model, don't spend time on it.

Principle 6. Adapt the models you develop to the system at hand. It may be necessary to adapt model notation or rules to the application; for example, a video game application might require a different modeling technique than real-time, embedded software that controls an automobile engine.

Principle 7. Try to build useful models, but forget about building perfect models. When building requirements and design models, a software engineer reaches a point of diminishing returns.

Principle 8. Don't become dogmatic about the syntax of the model. If it communicates content successfully, representation is secondary. Although everyone on a software team should try to use consistent notation during modeling, the most important characteristic of the model is to communicate information that enables the next software engineering task. If a model does this successfully, incorrect syntax can be forgiven.

Principle 9. If your instincts tell you a model isn't right even though it seems okay on paper, you probably have reason to be concerned. If experienced software engineer, trust instincts. Software work teaches many lessons, some of them on a subconscious level. If something says that a design model is doomed to fail have reason to spend additional time examining the model or developing a different one.

Principle 10. Get feedback as soon as you can. Every model should be reviewed by members of the software team. The intent of these reviews is to provide feedback that can be used to correct modeling mistakes, change misinterpretations, and add features or functions that were inadvertently omitted.

Requirements modeling principles.

VTUSYNC.IN

- ✓ Over the past three decades, many requirements modeling methods have been developed.
- ✓ Investigators have identified requirements analysis problems and their causes and have developed a variety of modeling notations and corresponding sets of heuristics to overcome them.
- ✓ Each analysis method has a unique point of view. However, all analysis methods are related by a set of operational principles:

Principle 1. The information domain of a problem must be represented and understood. The information domain encompasses the data that flow into the system (from end users, other systems, or external devices), the data that flow out of the system (via the user interface, network interfaces, reports, graphics, and other means), and the data stores that collect and organize persistent data objects (i.e., data that are maintained permanently).

Principle 2. The functions that the software performs must be defined. Software functions provide direct benefit to end users and provide internal support for those features that are user visible. Some functions transform data that flow into the system.

Principle 3. The behavior of the software (as a consequence of external events) must be represented. The behavior of computer software is driven by its interaction with the external environment. Input provided by end users, control data provided by an external system, or monitoring data collected over a network all cause the software to behave in a specific way.

Principle 4. The models that depict information, function, and behavior must be partitioned in a manner that uncovers detail in a layered (or hierarchical) fashion. Requirements modeling is the first step in software engineering problem solving. It allows you to better understand the problem and establishes a basis for the solution (design). Complex problems are difficult to solve in their entirety. For this reason, you should use a divide-and-conquer strategy. A large, complex problem is divided into subproblems until each subproblem is relatively easy to understand. This concept is called partitioning or separation of concerns, and it is a key strategy in requirements modeling.

Principle 5. The analysis task should move from essential information toward implementation detail. Requirements modeling begins by describing the problem from the end-user's perspective. The "essence" of the problem is described without any consideration of how a solution will be implemented.

Design Modeling Principles.

VTUSYNC.IN

- ✓ The software design model is analogous to an architect's plans for a house. It begins by representing the totality of the thing to be built (e.g., a three-dimensional rendering of the house) and slowly refines the thing to provide guidance for constructing each detail (e.g., the plumbing layout). Similarly, the design model that is created for software provides a variety of different views of the system.
- ✓ There is no shortage of methods for deriving the various elements of a software design. Some methods are data driven, allowing the data structure to dictate the program architecture and the resultant processing components.
- ✓ Others are pattern driven, using information about the problem domain (the requirements model) to develop architectural styles and processing patterns. Still others are object oriented, using problem domain objects as the driver for the creation of data structures and the methods that manipulate them.
- ✓ A set of design principles that can be applied regardless of the method that is used:

Principle 1. Design should be traceable to the requirements model. The requirements model describes the information domain of the problem, user-visible functions, system behavior, and a set of requirements classes that package business objects with the methods that service them. The design model translates this information into an architecture, a set of subsystems that implement major functions, and a set of components that are the realization of requirements classes. The elements of the design model should be traceable to the requirements model.

Principle 2. Always consider the architecture of the system to be built. Software architecture is the skeleton of the system to be built. It affects interfaces, data structures, program control flow and behavior, the way testing can be conducted, the maintainability of the resultant system, and much more. For all these reasons, design should start with architectural considerations. Only after the architecture has been established should component-level issues be considered.

Principle 3. Design of data is as important as design of processing functions. Data design is an essential element of architectural design. The way data objects are realized within the design cannot be left to chance. A well-structured data design helps to simplify program flow, makes the design and implementation of software components easier, and makes overall processing more efficient

Principle 4. Interfaces (both internal and external) must be designed with care. The way data flows between the components of a system has much to do with processing efficiency, error propagation, and design simplicity. A well-designed interface makes integration easier and assists the tester in validating component functions.

VTUSYNC.IN

Principle 5. User interface design should be tuned to the needs of the end user. However, in every case, it should stress ease of use. The user interface is the visible manifestation of the software. No matter how sophisticated its internal functions, no matter how comprehensive its data structures, no matter how well designed its architecture, a poor interface design often leads to the perception that the software is "bad."

Principle 6. Component-level design should be functionally independent. Functional independence is a measure of the "single-mindedness" of a software component. The functionality that is delivered by a component should be cohesive—that is, it should focus on one and only one function or subfunction.5

Principle 7. Components should be loosely coupled to one another and to the external environment. Coupling is achieved in many ways— via a component interface, by messaging, through global data. As the level of coupling increases, the likelihood of error propagation also increases and the overall maintainability of the software decreases. Therefore, component coupling should be kept as low as is reasonable.

Principle 8. Design representations (models) should be easily understandable. The purpose of design is to communicate information to practitioners who will generate code, to those who will test the software, and to others who may maintain the software in the future. If the design is difficult to understand, it will not serve as an effective communication medium.

Principle 9. The design should be developed iteratively. With each iteration, the designer should strive for greater simplicity. Like almost all creative activities, design occurs iteratively. The first iterations work to refine the design and correct errors, but later iterations should strive to make the design as simple as is possible.

VTUSYNC.IN

Construction Principles

- ✓ The construction activity encompasses a set of coding and testing tasks that lead to operational software that is ready for delivery to the customer or end user.
- ✓ In modern software engineering work, coding may be
  - i. the direct creation of programming language source code (e.g., Java),
  - ii. the automatic generation of source code using an intermediate design-like representation of the component to be built, or
  - iii. the automatic generation of executable code using a "fourth-generation programming language" (e.g., Visual C).
- ✓ The initial focus of testing is at the component level, often called unit testing. Other levels of testing include
  - i. integration testing (conducted as the system is constructed),

ii. validation testing that assesses whether requirements have been met for the complete system (or software increment), and
iii. acceptance testing that is conducted by the customer in an effort to exercise all required features and functions.

✓ The following set of fundamental principles and concepts are applicable to coding and testing:

Coding Principles.

✓ The principles that guide the coding task are closely aligned with programming style, programming languages, and programming methods. However, there are a number of fundamental principles that can be stated:

Preparation principles: Before you write one line of code, be sure on

- ❖ Understand of the problem you're trying to solve.
- ❖ Understand basic design principles and concepts.
- ❖ Pick a programming language that meets the needs of the software to be built and the environment in which it will operate.
- ❖ Select a programming environment that provides tools that will make your work easier.
- ❖ Create a set of unit tests that will be applied once the component you code is completed.

Programming principles: As you begin writing code, be sure on

- ❖ Constrain your algorithms by following structured programming practice.
- ❖ Consider the use of pair programming.
- ❖ Select data structures that will meet the needs of the design.
- ❖ Understand the software architecture and create interfaces that are consistent with it.
- ❖ Keep conditional logic as simple as possible.
- ❖ Create nested loops in a way that makes them easily testable.
- ❖ Select meaningful variable names and follow other local coding standards.
- ❖ Write code that is self-documenting.
- ❖ Create a visual layout (e.g., indentation and blank lines) that aids understanding.

Validation Principles: After you've completed your first coding pass, be sure on

- ❖ Conduct a code walkthrough when appropriate.
- ❖ Perform unit tests and correct errors you've uncovered.
- ❖ Refactor the code.

Testing Principles.

VTUSYNC.IN

- ✓ In a classic book on software testing, several rules that can serve well as testing objectives:
  - ❖ Testing is a process of executing a program with the intent of finding an error.
  - ❖ A good test case is one that has a high probability of finding an as-yetundiscovered error.
  - ❖ A successful test is one that uncovers an as-yet-undiscovered error.

Different testing principles are

Principle 1. All tests should be traceable to customer requirements. The objective of software testing is to uncover errors. It follows that the most severe defects (from the customer's point of view) are those that cause the program to fail to meet its requirements

Principle 2. Tests should be planned long before testing begins. Test planning can begin as soon as the requirements model is complete. Detailed definition of test cases can begin as soon as the design model has been solidified. Therefore, all tests can be planned and designed before any code has been generated

Principle 3. The Pareto principle applies to software testing. In this context the Pareto principle implies that 80 percent of all errors uncovered during testing will likely be traceable to 20 percent of all program components. The problem, of course, is to isolate these suspect components and to thoroughly test them

Principle 4. Testing should begin "in the small" and progress toward testing "in the large." The first tests planned and executed generally focus on individual components. As testing progresses, focus shifts to find errors in integrated clusters of components and ultimately in the entire system

Principle 5. Exhaustive testing is not possible. The number of path permutations for even a moderately sized program is exceptionally large. For this reason, it is impossible to execute every combination of paths during testing.

Deployment Principles

- ✓ Each delivery cycle provides the customer and end users with an operational software increment that provides usable functions and features.
- ✓ Each support cycle provides documentation and human assistance for all functions and features introduced during all deployment cycles to date.
- ✓ Each feedback cycle provides the software team with important guidance that results in modifications to the functions, features, and approach taken for the next increment.
- ✓ The delivery of a software increment represents an important milestone for any software project.

VTUSYNC.IN

- ✓ A number of key principles should be followed as the team prepares to deliver an increment:

Principle 1. Customer expectations for the software must be managed. Too often, the customer expects more than the team has promised to deliver, and disappointment occurs immediately. This results in feedback that is not productive and ruins team morale.

Principle 2. A complete delivery package should be assembled and tested. A CD-ROM or other media (including Web-based downloads) containing all executable software, support data files, support documents, and other relevant information should be assembled and thoroughly beta-tested with actual users. All installation scripts and other operational features should be thoroughly exercised in as many different computing configurations (i.e., hardware, operating systems, peripheral devices, networking arrangements) as possible.

Principle 3. A support regime must be established before the software is delivered. An end user expects responsiveness and accurate information when a question or problem arises. If support is ad hoc, or worse, nonexistent, the customer will become dissatisfied immediately. Support should be planned, support materials should be prepared, and appropriate record keeping mechanisms should be established so that the software team can conduct a categorical assessment of the kinds of support requested.

Principle 4. Appropriate instructional materials must be provided to end users. The software team delivers more than the software itself. Appropriate training aids (if required) should be developed; troubleshooting guidelines should be provided, and when necessary

Principle 5. Buggy software should be fixed first, delivered later. Under time pressure, some software organizations deliver low-quality increments with a warning to the customer that bug “will be fixed in the next release.”

VTUSYNC.IN